

REDES NEURONALES

1. OBJETIVOS

Objetivo Principal

- Realizar un programa para el entrenamiento de una red neuronal que simule el comportamiento de una compuerta XOR utilizando el algoritmo Backpropagation; observando el comportamiento de la red neuronal al realizar cambios tanto en el momentum(β) como en el factor de entrenamiento (α).

Objetivos Específicos

- Modificar el factor de entrenamiento (α) desde 0.1 hasta 2 con incrementos de 0.1, manteniendo constantes los pesos iniciales y momentum (β) igual a cero y observar su comportamiento mediante gráfica.
- Modificar el momentum (β) desde 0.0 hasta 1 con incrementos de 0.1, manteniendo constantes los pesos iniciales y un factor de entrenamiento (α) igual a 0.5 y observar su comportamiento mediante gráfica.
- Realizar modificaciones tanto en el factor de entrenamiento (α) como en el momentum y observar su comportamiento mediante grafica (β).

REDES NEURONALES

PROCEDIMIENTO

El objetivo de este laboratorio es realizar en MATLAB el algoritmo de entrenamiento para una RNA que simule el comportamiento de una compuerta XOR. Antes de empezar a detallar como se realizo el programa vale la pena aclarar que la red Backpropagation ó regla delta generalizada, se trata simplemente de un método descendente de gradiente, con el cual podemos reducir, minimizar el error cuadrático total de la salida calculada por la red.

Este método de entrenamiento se utiliza para entrenar redes multicapas feedforward, con funciones de transferencia diferenciables para ejecutar funciones de aproximación, asociación de patrones, y clasificación de patrones. Ahora procedemos a detallar los pasos a seguir para el desarrollo del algoritmo de entrenamiento. Inicialmente definimos las entradas de nuestra RNA y las salidas que se desean obtener para dichas entradas, en el caso de la XOR estas serian:

X1	X2	Yd
0	0	0
0	1	1
1	0	1
1	1	0

Tabla 1. Entradas y Salidas para la XOR.

Ahora lo que hacemos es calcular las salidas de las capas ocultas de la red (Ver **Figura 1.**)

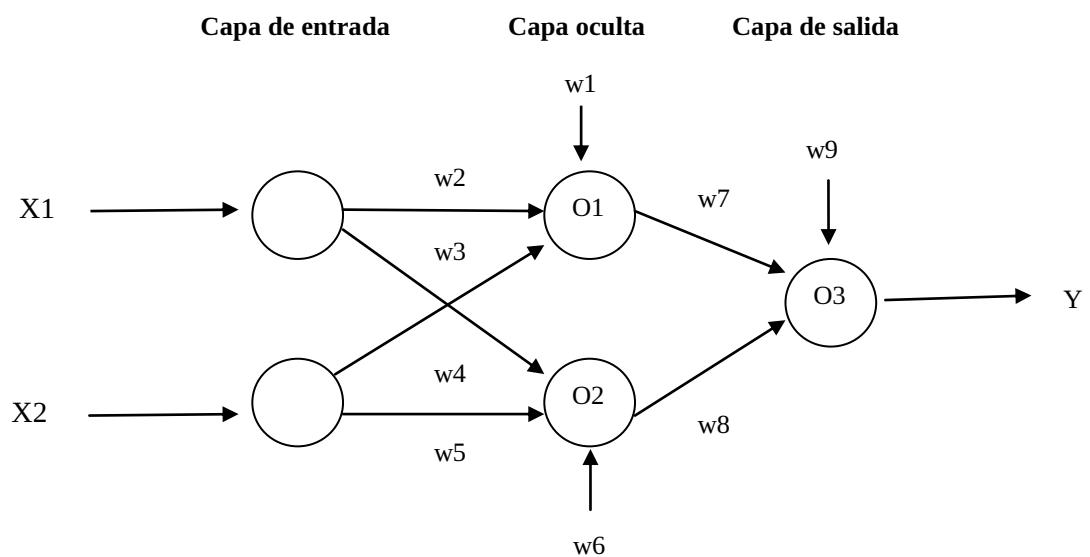


Figura 1. Esquema de la red neuronal

REDES NEURONALES

$$S1 = x_1w_2 + x_2w_4 + w_1$$

$$S2 = x_1w_3 + x_2w_5 + w_6$$

$$O_1 = F(S1)$$

$$O_2 = F(S2)$$

$$F = \frac{1}{1 + e^{-x}}$$

Donde F, es la función sigmoideal, la cual es totalmente derivable. Teniendo esto procedemos a calcular la salida Y de la red de la siguiente manera:

$$S3 = O_1w_7 + O_2w_8 + w_9$$

$$O = F(S3)$$

Ahora calculamos el error:

$$error = Yd - O$$

Ahora procedemos a propagar el error a las capas anteriores, para ello inicialmente calculamos δ_0 y hallamos la variación en los pesos 7, 8 y 9 con las siguientes formulas:

$$\delta_0 = error * F'(S3)$$

$$\Delta w_7 = \alpha \delta_0 O_1$$

$$\Delta w_8 = \alpha \delta_0 O_2$$

$$\Delta w_9 = \alpha \delta_0$$

Calculamos la variación en las salidas intermedias;

$$dS1 = \delta_0 w_7$$

$$dS2 = \delta_0 w_8$$

$$\delta_1 = dS1 F'(S1)$$

$$\delta_2 = dS2 F'(S2)$$

Teniendo esto hallamos las variaciones en los delta faltantes:

$$\Delta w_1 = \alpha \delta_1$$

$$\Delta w_2 = \alpha \delta_1 x_1$$

$$\Delta w_3 = \alpha \delta_2 x_1$$

$$\Delta w_4 = \alpha \delta_1 x_2$$

$$\Delta w_5 = \alpha \delta_2 x_2$$

$$\Delta w_6 = \alpha \delta_2$$

REDES NEURONALES

Teniendo las variaciones de los pesos, procedemos a re calcular los pesos con las siguientes formula:

$$w = w + \Delta w + \beta \Delta w_{ant}$$

Hacemos esto para los 4 casos y cuando realicemos lo anterior, calculamos el error cuadrático medio:

$$\varepsilon = \frac{1}{2}error^2 = \frac{1}{2}(error_1^2 + error_2^2 + error_3^2 + error_4^2)$$

Repetimos el proceso anterior actualizando los pesos hasta que el error cuadrático medio sea menor a un valor deseado. Una vez aclarado los pasos que debe tener nuestro programa, procedemos a realizar el algoritmo para una red Backpropagation en MATLAB como se muestra a continuación:

```
%% RED NEURONAL BACKPROPAGATION PARA UNA COMPUERTA XOR
%
*****

%
*****

clear all
clc

%Definimos variables
%Entradas X0, X1, X2
x0 = 1;
x1 = [0 0 1 1];
x2 = [0 1 0 1];
Yd = xor(x1,x2);

%Inicializamos los pesos aleatoriamente
wi = rand (1,9);
%wi = [0.50281 0.70947 0.42889 0.30462 0.18965 0.19343 0.68222
0.30276 0.54167];
w = wi;

for i=1:1:9
A = num2str(i);
eval(['wi' A '=w(i)']);
eval(['w' A '=w(i)']);
clc
end

%alpha = 0.1
```

REDES NEURONALES

```
%beta = 0.4
%error = 0.01;
%% Seleccion de los valores de alpha, beta y el error deseado
disp('-----')
disp('          RED NEURONAL BACKPROPAGATION COMPUERTA XOR')
disp('-----')
A = inputdlg ('Ingrese el valor de alpha deseado','Alpha: ');
alpha = eval(A{1});
B = inputdlg ('Ingrese el valor de beta deseado','Beta: ');
beta = eval(B{1});
C = inputdlg ('Ingrese el error deseado','Error: ');
E = eval(C{1});
clc
%% Algoritmo backpropagation

Y = zeros(1,4);
e = zeros(1,4);
aux = 1;
dwant = zeros(1,9);
counter = 0;
ec = 1;

while ec > E
    ec = 0;
    for i = 1:1:4
        S1 = (x1(i)*w2 + x2(i)*w4 + w1);
        S2 = (x1(i)*w3 + x2(i)*w5 + w6);
        O1 = (1/(1+exp(-S1)));
        O2 = (1/(1+exp(-S2)));
        S3 = (O1*w7+O2*w8+w9);
        O = (1/(1+exp(-S3)));
        Y(i) = O;
        error = Yd(i)-O;

        if error ~= 0
            d0 = error * (1/(1+exp(-S3))^2*exp(-S3));
            dw7 = alpha*d0*O1;
            dw8 = alpha*d0*O2;
            dw9 = alpha*d0;
            dS1 = d0*w7;
            dS2 = d0*w8;
            d1 = dS1* (1/(1+exp(-S1))^2*exp(-S1));
            d2 = dS2* (1/(1+exp(-S2))^2*exp(-S2));
            dw1 = alpha*d1;
            dw2 = alpha*d1*x1(i);
            dw3 = alpha*d2*x1(i);
            dw4 = alpha*d1*x2(i);
            dw5 = alpha*d2*x2(i);
            dw6 = alpha*d2;

            for k=1:1:9
                dw = [dw1 dw2 dw3 dw4 dw5 dw6 dw7 dw8 dw9];
                w(k) = w(k)+dw(k)+dwant(k)*beta;    %momentum
            end
            dwant = dw;
        end
    end
    counter = counter + 1;
    ec = ec + error^2;
end
```

REDES NEURONALES

```
w1 = w(1);
w2 = w(2);
w3 = w(3);
w4 = w(4);
w5 = w(5);
w6 = w(6);
w7 = w(7);
w8 = w(8);
w9 = w(9);

end
e(i) = error;
end
ec = (1/2)*sum(e.^2);
counter = counter + 1;

if counter >= 40000
    b = 1;
    break
else
    b = 0;
end
end
%% Resultados

if b ~= 1
    disp('-----')
    -')
    disp('Los Pesos sinapticos iniciales para la RNA son:')
    disp(['w1: ', num2str(w1)])
    disp(['w2: ', num2str(w2)])
    disp(['w3: ', num2str(w3)])
    disp(['w4: ', num2str(w4)])
    disp(['w5: ', num2str(w5)])
    disp(['w6: ', num2str(w6)])
    disp(['w7: ', num2str(w7)])
    disp(['w8: ', num2str(w8)])
    disp(['w9: ', num2str(w9)])
    disp('-----')
    -')
    -')
    disp('-----')
    -')
    disp('Los Pesos sinapticos finales para la RNA son:')
    disp(['w1: ', num2str(w1)])
    disp(['w2: ', num2str(w2)])
    disp(['w3: ', num2str(w3)])
    disp(['w4: ', num2str(w4)])
    disp(['w5: ', num2str(w5)])
    disp(['w6: ', num2str(w6)])
    disp(['w7: ', num2str(w7)])
    disp(['w8: ', num2str(w8)])
    disp(['w9: ', num2str(w9)])
    disp('-----')
    -')
    disp(['El numero de iteraciones para el entrenamiento de la XOR
es: ', num2str(counter)])
    disp('-----')
    -')
```

REDES NEURONALES

```
        disp('-----')
    -')
    disp(['La Salida deseada es: ',num2str(Yd)])
    disp(['La Salida obtenida es: ',num2str(Y)])
    disp('-----')
    -')

else
    disp('-----')
    -')
    disp('La red no pudo ser entrenada satisfactoriamente')
    disp(['para un alpha = ',num2str(alpha)])
    disp('Modifique el valor de alpha o los pesos iniciales')
    disp('-----')
    -')
end
```

Ahora procedemos a hacer la grafica de alpha Vs. Número de iteraciones, tomando unos pesos constantes, variando alpha de 0.1 hasta 2, incrementándose en pasos de 0.1, un valor de Beta fijo igual a cero y un error de 0.01. El programa que nos permite lograr tal objetivo se muestra a continuación:

```
%% RED NEURONAL BACKPROPAGATION PARA UNA COMPUERTA XOR VARIANDO LOS VALORES DE ALPHA
%
*****

*****
close all
clear all
clc

%Definimos variables
%Entradas X0, X1, X2
x0 = 1;
x1 = [0 0 1 1];
x2 = [0 1 0 1];
Yd = xor(x1,x2);

%Inicializamos los pesos
wi = [0.0844    0.3998    0.2599    0.8001    0.4314    0.9106
0.1818    0.2638    0.1455];
w = wi;

beta = 0;
E = 0.01;
%% Algoritmo backpropagation
```

REDES NEURONALES

```
contador = 0;
ciclos = zeros(1,20);

for alpha = 0.1:0.1:2
    contador = contador+1;
    w = wi;
    Y = zeros(1,4);
    e = zeros(1,4);
    aux = 1;
    dwant = zeros(1,9);
    counter = 0;
    ec = 1;

    for z=1:1:9
        A = num2str(z);
        eval(['wi' A '=-w(z)']);
        eval(['w' A '=-w(z)']);
        clc
    end

    while ec > E
        ec = 0;
        %counter = counter + 1;
        for i = 1:1:4
            S1 = (x1(i)*w2 + x2(i)*w4 + w1);
            S2 = (x1(i)*w3 + x2(i)*w5 + w6);
            O1 = (1/(1+exp(-S1)));
            O2 = (1/(1+exp(-S2)));
            S3 = (O1*w7+O2*w8+w9);
            O = (1/(1+exp(-S3)));
            Y(i) = O;
            error = Yd(i)-O;

            if error ~= 0
                d0 = error * (1/(1+exp(-S3))^2*exp(-S3));
                dw7 = alpha*d0*O1;
                dw8 = alpha*d0*O2;
                dw9 = alpha*d0;
                dS1 = d0*w7;
                dS2 = d0*w8;
                d1 = dS1* (1/(1+exp(-S1))^2*exp(-S1));
                d2 = dS2* (1/(1+exp(-S2))^2*exp(-S2));
                dw1 = alpha*d1;
                dw2 = alpha*d1*x1(i);
                dw3 = alpha*d2*x1(i);
                dw4 = alpha*d1*x2(i);
                dw5 = alpha*d2*x2(i);
                dw6 = alpha*d2;

                for k=1:1:9
                    dw = [dw1 dw2 dw3 dw4 dw5 dw6 dw7 dw8 dw9];
                    w(k) = w(k)+dw(k)+dwant(k)*beta;    %momentum
                end
                dwant = dw;
                w1 = w(1);
                w2 = w(2);
                w3 = w(3);
                w4 = w(4);
            end
        end
    end
end
```

REDES NEURONALES

```
        w5 = w(5);
        w6 = w(6);
        w7 = w(7);
        w8 = w(8);
        w9 = w(9);
    end
    e(i) = error;
end
ec = (1/2)*sum(e.^2);
counter = counter + 1;

if counter >= 15000
    b = 1;
    break
else
    b = 0;
end
end
ciclos(contador)=counter;
end
alpha = 0.1:0.1:2;
plot(alpha,ciclos);
grid on;
title('GRÁFICA DE LA RED BACKPROPAGATION VARIANDO LOS VALORES DE
ALPHA');
xlabel('alpha');
ylabel('Numero de iteraciones');
```

Luego realizamos la grafica de beta Vs. Número de iteraciones, tomando unos pesos constantes, variando beta de 0 hasta 1, incrementándose en pasos de 0.1, un valor de alpha fijo igual a 0.5 y un error de 0.01. El programa que nos permite lograr tal objetivo se muestra a continuación:

```
%% RED NEURONAL BACKPROPAGATION PARA UNA COMPUERTA XOR VARIANDO BETA
%
*****

*****
close all
clear all
clc

%Definimos variables
%Entradas X0, X1, X2
x0 = 1;
x1 = [0 0 1 1];
x2 = [0 1 0 1];
Yd = xor(x1,x2);
```

REDES NEURONALES

```
%Inicializamos los pesos
wi = [0.0844    0.3998    0.2599    0.8001    0.4314    0.9106
0.1818    0.2638    0.1455];
w = wi;

alpha = 0.5
E = 0.01;
%% Algoritmo backpropagation

contador = 0;
ciclos = zeros(1,10);

for beta = 0:0.1:1
    contador = contador+1;
    w = wi;
    Y = zeros(1,4);
    e = zeros(1,4);
    aux = 1;
    dwant = zeros(1,9);
    counter = 0;
    ec = 1;

    for z=1:1:9
        A = num2str(z);
        eval(['wi' A ' = w(z) ']);
        eval(['w' A ' = w(z) ']);
        clc
    end

    while ec > E
        ec = 0;
        %counter = counter + 1;
        for i = 1:1:4
            S1 = (x1(i)*w2 + x2(i)*w4 + w1);
            S2 = (x1(i)*w3 + x2(i)*w5 + w6);
            O1 = (1/(1+exp(-S1)));
            O2 = (1/(1+exp(-S2)));
            S3 = (O1*w7+O2*w8+w9);
            O = (1/(1+exp(-S3)));
            Y(i) = O;
            error = Yd(i)-O;

            if error ~= 0
                d0 = error * (1/(1+exp(-S3))^2*exp(-S3));
                dw7 = alpha*d0*O1;
                dw8 = alpha*d0*O2;
                dw9 = alpha*d0;
                dS1 = d0*w7;
                dS2 = d0*w8;
                d1 = dS1* (1/(1+exp(-S1))^2*exp(-S1));
                d2 = dS2* (1/(1+exp(-S2))^2*exp(-S2));
                dw1 = alpha*d1;
                dw2 = alpha*d1*x1(i);
                dw3 = alpha*d2*x1(i);
                dw4 = alpha*d1*x2(i);
                dw5 = alpha*d2*x2(i);
```

REDES NEURONALES

```
dw6 = alpha*d2;

for k=1:1:9
    dw = [dw1 dw2 dw3 dw4 dw5 dw6 dw7 dw8 dw9];
    w(k) = w(k)+dw(k)+dwant(k)*beta;    %momentum
end
dwant = dw;
w1 = w(1);
w2 = w(2);
w3 = w(3);
w4 = w(4);
w5 = w(5);
w6 = w(6);
w7 = w(7);
w8 = w(8);
w9 = w(9);
end
e(i) = error;
end
ec = (1/2)*sum(e.^2);
counter = counter + 1;

if counter >= 15000
    b = 1;
    break
else
    b = 0;
end
end
ciclos(contador)=counter;
end
beta = 0:0.1:1;
plot(beta,ciclos);
grid on;
title('GRÁFICA DE LA RED BACKPROPAGATION VARIANDO LOS VALORES DE
BETA');
xlabel('beta');
ylabel('Numero de iteraciones');
```

Finalmente realizamos un grafica en 3D variando los valores de alpha y Beta con respecto al número de iteraciones, el programa que cumple este objetivo se muestra a continuación:

```
%% RED NEURONAL BACKPROPAGATION PARA UNA COMPUERTA XOR VARIANDO
ALPHA Y BETA
%
*****

*****
clear all
clc
```

REDES NEURONALES

```
%Definimos variables
%Entradas X0, X1, X2
x0 = 1;
x1 = [0 0 1 1];
x2 = [0 1 0 1];
Yd = xor(x1,x2);

%Inicializamos los pesos
wi = [0.0844    0.3998    0.2599    0.8001    0.4314    0.9106
0.1818    0.2638    0.1455];
w = wi;
%E = 0.01;
%% Seleccion del error deseado
C = inputdlg ('Ingrese el error deseado','Error: ');
E = eval(C{1});
clc
%% Algoritmo backpropagation

ALPHA =[];
BETA =[];
ITERACIONES =[];
CONT1=0;
CONT2=0;

for alpha = 0.1:0.1:2
    CONT1 =CONT1+1;
    CONT2 = 0;

    for beta = 0:0.1:1
        CONT2 = CONT2+1;
        w = wi;
        Y = zeros(1,4);
        e = zeros(1,4);
        dwant = zeros(1,9);
        counter = 0;
        ec = 1;

        for z=1:1:9
            A = num2str(z);
            eval(['wi' A ' '=w(z) ']);
            eval(['w' A ' '=w(z) ']);
            clc
        end

        while ec > E
            ec = 0;
            for i = 1:1:4
                S1 = (x1(i)*w2 + x2(i)*w4 + w1);
                S2 = (x1(i)*w3 + x2(i)*w5 + w6);
                O1 = (1/(1+exp(-S1)));
                O2 = (1/(1+exp(-S2)));
                S3 = (O1*w7+O2*w8+w9);
                O = (1/(1+exp(-S3)));
                Y(i) = O;
                error = Yd(i)-O;
            end
        end
    end
end
```

REDES NEURONALES

```
if error ~= 0
    d0 = error * (1/(1+exp(-S3))^2*exp(-S3));
    dw7 = alpha*d0*O1;
    dw8 = alpha*d0*O2;
    dw9 = alpha*d0;
    dS1 = d0*w7;
    dS2 = d0*w8;
    d1 = dS1* (1/(1+exp(-S1))^2*exp(-S1));
    d2 = dS2* (1/(1+exp(-S2))^2*exp(-S2));
    dw1 = alpha*d1;
    dw2 = alpha*d1*x1(i);
    dw3 = alpha*d2*x1(i);
    dw4 = alpha*d1*x2(i);
    dw5 = alpha*d2*x2(i);
    dw6 = alpha*d2;

    for k=1:1:9
        dw = [dw1 dw2 dw3 dw4 dw5 dw6 dw7 dw8 dw9];
        w(k) = w(k)+dw(k)+dwant(k)*beta;    %momentum
    end

    dwant = dw;
    w1 = w(1);
    w2 = w(2);
    w3 = w(3);
    w4 = w(4);
    w5 = w(5);
    w6 = w(6);
    w7 = w(7);
    w8 = w(8);
    w9 = w(9);
end
e(i) = error;
end
ec = (1/2)*sum(e.^2);
counter = counter + 1;

if counter >= 1000
    b = 1;
break
else
    b = 0;
end
end

ALPHA(CONT1,CONT2) = alpha;
BETA(CONT1,CONT2) = beta;
ITERACIONES(CONT1,CONT2) = counter;
end

plot3(ALPHA,BETA,ITERACIONES);
grid on;
title('GRÁFICA DE LA RED BACKPROPAGATION VARIANDO LOS VALORES DE
ALPHA Y BETA');
xlabel('alpha');
ylabel('beta');
zlabel ('Número de iteraciones');
```